

**SUPERVISED MACHINE LEARNING**  
**Simple Linear Regression**

**Jean René RANDRIANJAKAMANANA**

2024-11-25

# Table de matières

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>SUPERVISED MACHINE LEARNING</b>                      | <b>2</b> |
| <b>I) REGRESSION WITH ONE FEATURE (X)</b>               | <b>3</b> |
| A) SIMPLE LINEAR REGRESSION $y = ax+b$ . . . . .        | 3        |
| 1) Model ameliorated through Gradient Descent . . . . . | 3        |
| 2) Model with sklearn . . . . .                         | 7        |

# **SUPERVISED MACHINE LEARNING**

# I) REGRESSION WITH ONE FEATURE (X)

The four (04) fundamental steps in **Supervised ML** are:

1. **DATASET**: **Target**: **Y** et **Features**: **X**
2. **MODELS**: **Model's parameters** (Linear, polynomial, exponential).
3. **LOSS FUNCTION**: Find the loss function  $e = (y - \hat{y})$ .
4. **ALGORITHM OF MINIMISATION**: Try to minimise the error (loss function) in order to have a better model.

## A) SIMPLE LINEAR REGRESSION $y = ax+b$

### 1) Model ameliorated through Gradient Descent

- A linear regression model is formed as:  $Y = X\beta + \varepsilon$  where  $X$  is a matrix of explicatives variables (feature) and  $Y$  the target one.
- The form of adjusted model that we have to build is:  $\hat{Y} = X\hat{\beta}$
- In the theory, we estimate the value of  $\beta$  by using the method of **Least Square** so we get  $\hat{\beta} = (X'X)^{-1} X'Y$ , with  $X'$  is the transpose of the matrix  $X$ .
- In the practical, we often use **the method of Gradient Descent** to estimate  $\beta$ . That method is based in the following formula:

Let  $J(\beta) = \frac{1}{2m} \|Y - \hat{Y}\|^2$ , the value of  $\hat{\beta}$  is :

$$\hat{\beta}_{n+1} = \hat{\beta}_n - \alpha \times \nabla J(\beta) \quad \text{with } \nabla J(\beta) = \vec{grad}J(\beta)$$

By using the package **sklearn** with the help of **sklearn.LinearRegression()**, we have the following steps:

- **Dataset**: Import of random data which consists of  $x$  and  $y$ .
- **Random parameters**: We create  $\beta$  (how we like it) randomly and we add constants in the matrix  $x$ . I.e.: we get  $X = [x, one]$  where  $one$  is composed only by one(1).
- **Models**: We calculate the matrix product  $X.\beta$ .
- Creation of the loss function or error function  $J(\beta)$ .

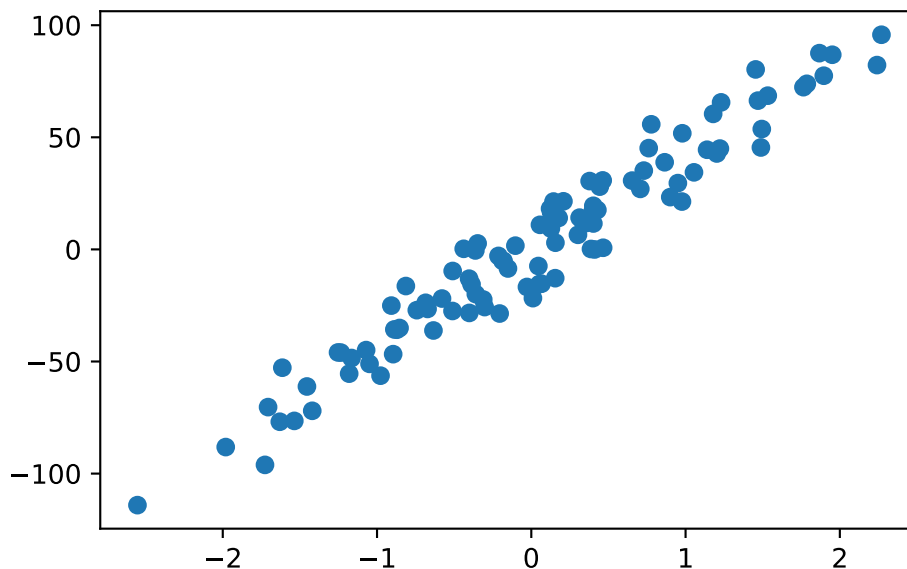
- Then here, the most important thing is to minimise the loss function by using the algorithm of gradient descent. That way we should have the better estimation of  $\beta$ . A good result will be got by manipulating the step of the gradient.
- The predicted value  $y_{pred}$  can be obtained through the result model got at the end.
- Plot in the same box  $A_i(x_i, y_i)$  and  $y_{pred_i} = f(x)$ .

However, `sklearn` with **LinearRegression** can be a shortcut to the final step.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from sklearn.datasets import make_regression
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error, r2_score
```

#### a) Importation of datasets and plot

```
1 np.random.seed(0)
2 x, y = make_regression(n_samples = 100, n_features = 1, noise = 10)
3 plt.close()
4 plt.scatter(x, y)
5 plt.show()
```



#### b) Dimension and resize of data

```
1 x.shape
2 y.shape
3 y = y.reshape(y.shape[0], 1)
4 y.shape
```

(100, 1)

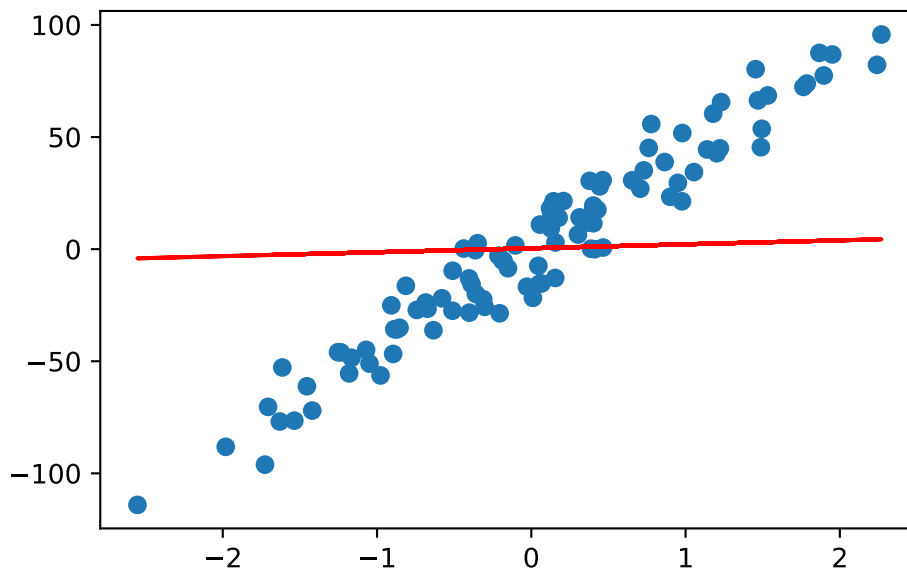
c) Random estimation of the model's parameters

```
1 np.random.seed(0)
2 X = np.hstack((x, np.ones(x.shape))) # Add the constant matrix
3 beta = np.random.randn(2,1)
4 beta
```

```
array([[1.76405235],
       [0.40015721]])
```

d) Model function creation and Regression plot

```
1 # Y = X*beta + epsilon #Y_ajust = X*beta_ajustr
2
3 def modele(X,beta):
4     return X.dot(beta)
5
6 plt.close()
7 plt.scatter(x,y)
8 plt.plot(x, modele(X, beta), c='r')
9 plt.show()
```



e) Loss function creation

```
1 def cost_function(X, y, beta):
2     m = len(y)
3     return (1/2*m)*np.sum((modele(X,beta) - y)**2)
4
5 cost_function(X,y,beta)
```

9056306.841935502

#### f) Minimisation of loss function: Gradient Descent

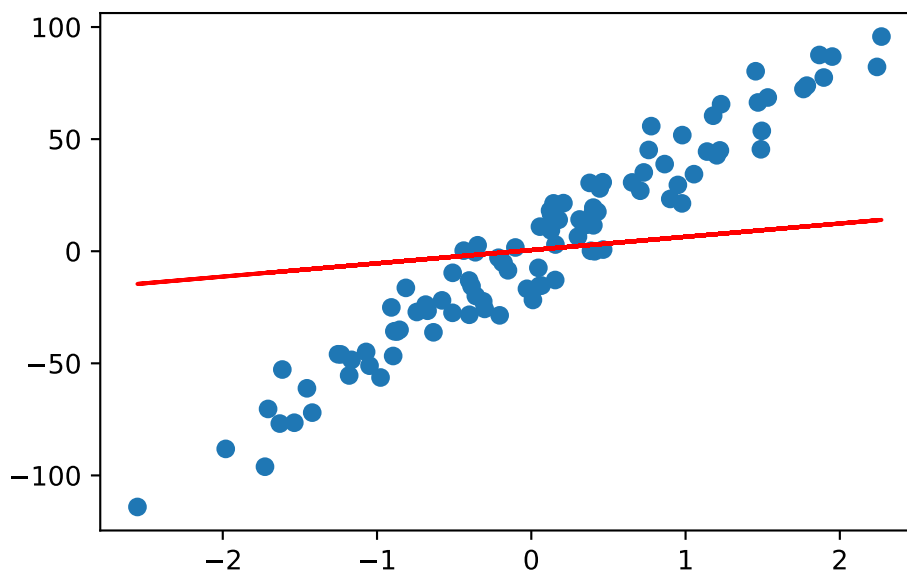
```
1 #Gradient function
2 def grad_cost_func(X, y, beta):
3     m = len(y)
4     return (1/m)*(X.T).dot(modele(X,beta) - y)
5
6 #Gradient descent algorithm
7 def gradient_descent(X,y,beta, learning_rate, n_iterations):
8     cost_history = np.zeros(n_iterations)
9     for i in range(0, n_iterations):
10         beta = beta - learning_rate*grad_cost_func(X,y,beta)
11         cost_history[i] = cost_function(X,y,beta)
12     return beta, cost_history
```

#### g) Model training

```
1 beta_final,cost_history = gradient_descent(X,y,beta, learning_rate = 0.1, n_iterations = 1000)
2 beta_final
```

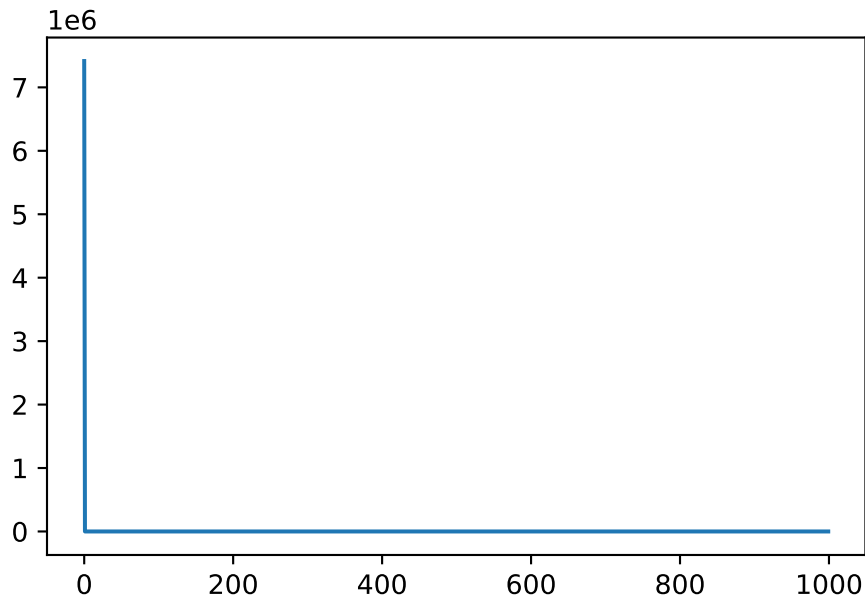
```
array([[5.92160163],
       [0.52307121]])
```

```
1 y_predi = modele(X, beta_final)
2 plt.close()
3 plt.scatter(x, y)
4 plt.plot(x, y_predi, c='r')
5 plt.show()
```



#### h) Learning curve (Decreasing plot of the loss function)

```
1 plt.close()
2 plt.plot(range(1000), cost_history)
3 plt.show()
```



Gradient descent algorithm found the optimal of the loss function at 400 iterations.

#### i) Coefficient of determination (R2)

```
1 def coeff_determination(y, y_predi):
2     u = ((y-y_predi)**2).sum()
3     v = ((y-np.mean(y))**2).sum()
4     return (1-u/v)
5
6 R2 = coeff_determination(y, y_predi)
7 print(f"R2 = {R2} or {round(R2,4)*100} %")
```

R2 = 0.24313519609213008 or 24.310000000000002 %

#### 2) Model with sklearn

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error, r2_score
5
6 # Random data
7 np.random.seed(0)
8 x, y = make_regression(n_samples = 100, n_features = 1, noise = 10)
9
```



```

10 # Object initiation
11 regression = LinearRegression()
12
13 #Training
14 regression.fit(x,y)
15 regression.coef_
16 regression.intercept_
17
18 #Prediction
19 y_pred = regression.predict(x)
20
21 #Model evaluation
22 R2 = regression.score(x,y)
23 rmse = mean_squared_error(y, y_pred)
24
25 # Values printing
26 print(f"Slope: {regression.coef_}\n ")
27 print(f"Intercept: {regression.intercept_}\n RMSE: {rmse} \n R2: {R2}")
28
29 # Plot of original data
30 plt.close()
31 plt.scatter(x,y, s=10)
32 plt.xlabel('x')
33 plt.ylabel('y')
34 plt.show()
35
36 #Predicted value
37 plt.close()
38 plt.scatter(x,y, s=10)
39 plt.plot(x, y_pred, color = 'r')
40 plt.xlabel('x')
41 plt.ylabel('y')
42 plt.show()

```

Slope: [42.61943029]

Intercept: -0.8141818270307257

RMSE: 114.17148616819482

R2: 0.9417294727711081

